

Animation 3D

Ajouter des animations 3D à votre page Web peut capter rapidement l'attention des utilisateurs et rendre votre site Web plus interactif et attrayant. Il existe de nombreuses bibliothèques JavaScript qui permettent de créer des animations 3D, telles que Three.js, Babylon.js, A-Frame, etc. Ces bibliothèques facilitent la création d'animations 3D complexes en utilisant WebGL, une API graphique 3D pour le navigateur Web.

Three.js

Three.js est une bibliothèque JavaScript qui permet de créer des animations 3D, des jeux et des expériences interactives en utilisant WebGL. Elle simplifie la création d'animations 3D en fournissant des objets prêts à l'emploi pour les caméras, les lumières, les matériaux, les géométries, etc. Three.js est largement utilisée pour créer des animations 3D sur le Web en raison de sa facilité d'utilisation et de sa puissance. Vous pouvez également combiner Three.js avec Motion pour créer des animations lors du défilement.

<https://threejs.org/docs/index.html#manual/en/introduction/Installation>

Installation de Three.js

Pour utiliser Three.js dans votre projet, vous pouvez l'installer via npm ou le charger directement depuis un lien CDN. Voici comment l'installer via npm :

```
npm install three
```

Créer une scène 3D avec Three.js (JavaScript vanille)

Une scène 3D est composée d'une scène, d'une caméra, d'un moteur de rendu et d'objets. Voici la version la plus simple en JavaScript pur, sans React.

HTML minimal (CDN)

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <title>Three.js - Scène de base</title>
    <!-- Three.js via CDN (objet global THREE) -->
    <script src="https://unpkg.com/three@0.160.0/build/three.min.js"></script>
    <style>
      html,
      body {
        margin: 0;
        height: 100%;
      }
      #app {
        width: 100vw;
        height: 100vh;
      }
    </style>
  </head>
  <body>
    <div id="app">
      <!-- Contenu de la scène 3D -->
    </div>
  </body>
</html>
```

```

        display: block;
    }
    /* Le canvas sera inséré dans #app */
</style>
</head>
<body>
    <div id="app"></div>
    <script src="./scene.js"></script>
</body>
</html>

```

JavaScript (scene.js)

```

// Récupérer le conteneur
const container = document.getElementById("app");

// 1) Scène
const scene = new THREE.Scene();

// 2) Caméra (FOV, aspect, near, far)
const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 1000);
camera.position.z = 5; // reculer la caméra pour voir l'objet

// 3) Rendu (la balise canvas est créée automatiquement)
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
container.appendChild(renderer.domElement);

// 4) Un objet simple (cube)
const geometry = new THREE.BoxGeometry();
const material = new THREE.MeshBasicMaterial({ color: 0x00ff00 });
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

// 5) Animation
function animate() {
    requestAnimationFrame(animate);
    cube.rotation.x += 0.01;
    cube.rotation.y += 0.01;
    renderer.render(scene, camera);
}
animate();

// 6) S'adapter au redimensionnement
window.addEventListener("resize", () => {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
});

```

Animer au défilement (vanilla JS)

On peut faire varier une propriété (ex: rotation) selon la progression du scroll. Ici, on calcule `scrollYProgress` en pourcentage de la page.

HTML (page plus haute pour scroller)

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <title>Three.js - Scroll</title>
    <script src="https://unpkg.com/three@0.160.0/build/three.min.js"></script>
    <style>
      body {
        margin: 0;
      }
      .spacer {
        height: 200vh;
      } /* crée du scroll */
      #app {
        position: fixed;
        inset: 0;
      } /* la scène reste visible */
    </style>
  </head>
  <body>
    <div id="app"></div>
    <div class="spacer"></div>
    <script src="./scroll-scene.js"></script>
  </body>
</html>
```

JavaScript (scroll-scene.js)

```
const container = document.getElementById("app");
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 1000);
camera.position.z = 5;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
container.appendChild(renderer.domElement);

const cube = new THREE.Mesh(new THREE.BoxGeometry(), new THREE.MeshBasicMaterial({
color: 0x00ff00 }));
scene.add(cube);

// Calculer la progression du scroll (0 -> 1)
function getScrollProgress() {
```

```

const scrollTop = window.scrollY;
const docHeight = document.documentElement.scrollHeight - window.innerHeight;
return docHeight > 0 ? scrollTop / docHeight : 0;
}

function animate() {
  requestAnimationFrame(animate);
  const p = getScrollProgress();
  cube.rotation.x = p * Math.PI * 2; // 1 tour complet sur X
  cube.rotation.y += 0.01; // rotation libre sur Y pour le fun
  renderer.render(scene, camera);
}
animate();

window.addEventListener("resize", () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});

```

Débogage (lil-gui) en vanilla JS

lil-gui fournit une petite interface pour ajuster des valeurs en temps réel.

HTML (CDN)

```

<script src="https://unpkg.com/three@0.160.0/build/three.min.js"></script>
<script src="https://unpkg.com/lil-gui@0.19/dist/lil-gui.umd.js"></script>
<div id="app"></div>
<script src="./debug-scene.js"></script>

```

JavaScript (debug-scene.js)

```

const container = document.getElementById("app");
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 1000);
camera.position.z = 5;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
container.appendChild(renderer.domElement);

const material = new THREE.MeshBasicMaterial({ color: 0x00ff00, wireframe: false
});
const cube = new THREE.Mesh(new THREE.BoxGeometry(), material);
scene.add(cube);

function animate() {
  requestAnimationFrame(animate);

```

```

        renderer.render(scene, camera);
    }
    animate();

    // GUI
    const gui = new lil.GUI();
    gui.add(cube.rotation, "x", -Math.PI, Math.PI, 0.01).name("Rotation X");
    gui.add(cube.rotation, "y", -Math.PI, Math.PI, 0.01).name("Rotation Y");
    gui.addColor({ color: "#00ff00" }, "color")
        .name("Couleur")
        .onChange((v) => {
            material.color.set(v);
        });
    gui.add(material, "wireframe").name("Wireframe");

    window.addEventListener("resize", () => {
        camera.aspect = window.innerWidth / window.innerHeight;
        camera.updateProjectionMatrix();
        renderer.setSize(window.innerWidth, window.innerHeight);
    });

```

Ajouter une texture à un objet 3D (vanilla)

On peut mapper une image sur un cube à l'aide de `TextureLoader`.

HTML

```

<script src="https://unpkg.com/three@0.160.0/build/three.min.js"></script>
<div id="app"></div>
<script src="./texture-scene.js"></script>

```

JavaScript (texture-scene.js)

```

const container = document.getElementById("app");
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
window.innerHeight, 0.1, 1000);
camera.position.z = 3;

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
container.appendChild(renderer.domElement);

const geometry = new THREE.BoxGeometry(1, 1, 1);
const loader = new THREE.TextureLoader();
// Remplacez par le chemin réel de votre image
const diffuse = loader.load("./images/texture.jpg");
const material = new THREE.MeshBasicMaterial({ map: diffuse });
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);

```

```
function animate() {  
    requestAnimationFrame(animate);  
    cube.rotation.x += 0.01;  
    cube.rotation.y += 0.01;  
    renderer.render(scene, camera);  
}  
animate();  
  
window.addEventListener("resize", () => {  
    camera.aspect = window.innerWidth / window.innerHeight;  
    camera.updateProjectionMatrix();  
    renderer.setSize(window.innerWidth, window.innerHeight);  
});
```

Références

<https://threejs-journey.com/>